

Linear Quadratic Regulator (LQR)

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 17, 2026

1. LQR Problem Formulation

Objective: Control a linear system to minimize a quadratic cost over a horizon N .

Discrete-time Linear Dynamics:

$$x_{t+1} = Ax_t + Bu_t, \quad x_0 \text{ given}$$

Quadratic Cost Function:

$$J(U) = \sum_{t=0}^{N-1} (x_t^T Q x_t + u_t^T R u_t) + x_N^T Q_f x_N$$

Weight Matrices:

- $Q \succeq 0, Q_f \succeq 0$: Penalize state deviation (Performance).
- $R \succ 0$: Penalize control effort (Energy).
- *Trade-off*: Large $Q \rightarrow$ Aggressive control, Large $R \rightarrow$ Smooth/Lazy control.

2. Approach 1: Batch Least Squares

We can view LQR as a giant static optimization problem. Stack all states and inputs:

$$X = [x_0, \dots, x_N]^T, U = [u_0, \dots, u_{N-1}]^T.$$

Linear Constraints: $X = GU + Hx_0$ (Dynamics matrix form) **Cost:** $J(U) = \|\bar{Q}X\|^2 + \|\bar{R}U\|^2$

Substituting X :

$$J(U) = \|\bar{Q}(GU + Hx_0)\|^2 + \|\bar{R}U\|^2$$

This is a standard ****Least Squares**** problem!

Pros/Cons:

- Easy to formulate.
- Computational Complexity: $O(N^3(n+m)^3)$ due to matrix inversion.
- **Too slow** for long horizons (Large N).

3. Approach 2: Dynamic Programming (Recursive)

DP exploits the sequential structure to solve LQR in $O(N)$.

Value Function Definition:

$$V_t(x) = \min_{u_t \dots u_{N-1}} \sum_{k=t}^{N-1} (x_k^T Q x_k + u_k^T R u_k) + x_N^T Q_f x_N$$

Terminal Condition:

$$V_N(x) = x^T Q_f x \triangleq x^T P_N x \quad (\text{where } P_N = Q_f)$$

Recursive Bellman Equation:

$$V_t(x) = \min_u [x^T Q x + u^T R u + V_{t+1}(Ax + Bu)]$$

4. Deriving the Riccati Equation (Blackboard)

Hypothesis: $V_{t+1}(x) = x^T P_{t+1} x$. Is $V_t(x)$ also quadratic?

Substitute into Bellman Eq:

$$V_t(x) = x^T Q x + \min_u [u^T R u + (A x + B u)^T P_{t+1} (A x + B u)]$$

Expand the term inside min:

$$(\dots) = u^T (R + B^T P_{t+1} B) u + 2x^T A^T P_{t+1} B u + x^T A^T P_{t+1} A x$$

Optimal Control u_t^* : Take gradient w.r.t u and set to 0.

$$2(R + B^T P_{t+1} B) u + 2B^T P_{t+1} A x = 0$$

$$u_t^* = - \underbrace{(R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A}_{K_t} x_t$$

5. The Riccati Recursion

Substitute $u_t^* = K_t x$ back into the Value Function equation:

$$V_t(x) = x^T \underbrace{[Q + A^T P_{t+1} A - A^T P_{t+1} B (R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A]}_{P_t} x$$

Discrete-time Riccati Equation (Backward Pass):

$$P_t = Q + A^T P_{t+1} A + A^T P_{t+1} B K_t$$

- We compute $P_N, P_{N-1}, \dots, P_0$ backwards.
- P_t is always symmetric and positive semi-definite ($P_t \succeq 0$) if $Q, Q_f \succeq 0$.

6. Summary: LQR Algorithm

Offline Phase (Backward Pass):

- ① Initialize $P_N = Q_f$.
- ② For $t = N - 1$ down to 0:
 - Compute Gain: $K_t = -(R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A$
 - Update Cost-to-Go Matrix: $P_t = Q + K_t^T R K_t + (A + B K_t)^T P_{t+1} (A + B K_t)$

Online Phase (Forward Pass):

- ① Given current state x_t .
- ② Compute Control: $u_t = K_t x_t$.
- ③ Apply to system: $x_{t+1} = A x_t + B u_t$.

Note: LQR provides a linear Feedback Policy, not just a trajectory.

7. Infinite Horizon LQR (Steady State)

What if $N \rightarrow \infty$? (Regulation for a long time)

- The matrix P_t converges to a steady-state value P_{ss} .
- P_{ss} satisfies the **Discrete Algebraic Riccati Equation (DARE)**:

$$P_{ss} = Q + A^T P_{ss} A - A^T P_{ss} B (R + B^T P_{ss} B)^{-1} B^T P_{ss} A$$

- The gain becomes constant: $K_{ss} = -(R + B^T P_{ss} B)^{-1} B^T P_{ss} A$.
- **Benefit:** No need to re-compute K_t at every step. Just multiply a constant matrix.

8. Extension: Tracking Control (Inhomogeneous Cost)

Goal: Track a reference trajectory $\bar{x}_t, \bar{u}_t$ instead of regulating to zero.

$$J = \sum (x_t - \bar{x}_t)^T Q (x_t - \bar{x}_t) + (u_t - \bar{u}_t)^T R (u_t - \bar{u}_t)$$

Trick: Augmented State Define $\tilde{x}_t = [x_t^T, 1]^T$. The cost becomes quadratic in $\tilde{x}$:

$$(x - \bar{x})^T Q (x - \bar{x}) = x^T Q x - 2\bar{x}^T Q x + \bar{x}^T Q \bar{x}$$

This looks like $\tilde{x}^T \tilde{Q} \tilde{x}$ where $\tilde{Q}$ includes cross-terms.

Result:

$$u_t = K_t x_t + k_t$$

We get a feedback term ($K_t x_t$) plus a feedforward term (k_t) derived from the reference.